

Week-1 DV Ramp Up Plan

Group-wise Topic Distribution

Group	Design	Students
Group A	UART	
Group B	PS/2	
Group C	SPI	

Work Expectation

Step	Activity	Description	Output
1	Learning	Understand concepts deeply	Notes
2	Design	Write RTL	SystemVerilog
3	Exploration	Analyze behavior	Logs
4	Output Generation	Structured outputs	JSON
5	Version Control	Track work	Git
6	Issue Analysis	Document problems	Issue Log

Presentation Guidelines & Structure Examples

❖ What Each Group Must Present

- Explain the core design and features
- Understand the internal logic and dataflow
- Think like a verification engineer about the design
- Show git repository structure
- Demonstrate JSON outputs
- Explain Approach and workflow
- Highlight Challenges and fixes

❖ Presentation Expectations

- Keep explanations simple and logical
- Focus on understanding, not memorization
- Use examples and snippets for your own work
- Be ready to answer questions from any part of your topic
- Every member should be able to explain the group work

❖ Examples of What You Will Be Asked

- Why did you design it this way?
- What was the reason for choosing this instead of something else?
- What happens if this fails?
- What assumptions did you make?
- What part was most difficult?

❖ Output Requirement

All results must be stored in structured JSON format, including:

- Design details
- Signals
- Operations
- Testcases
- Results

❖ Outcome Expectation of Week-1

Students should be able to :

- Design a complete ALU in systemVerilog
- Understand the RTL deeply
- Write Basic RTL for groupwise protocols
- Think in terms of Verification
- Understand protocol based designs
- Create structures testcases
- Perform basic verification
- Write simple testbench
- Maintain a proper git repository and json outputs

❖ Example of JSON Output Structure

```
{  
  "operation": "ADD",  
  "inputs": { "a": 10, "b": 5 },  
  "expected": 15,  
  "actual": 15,  
  "status": "PASS"  
}
```

❖ Example of Git Repository Structure

```
dv-ramp-up-week1/  
|  
├─ rtl/  
|   ├─ alu.sv  
|   └─ uart.sv / ps2.sv / spi.sv  
|  
├─ tb/  
|   ├─ alu_tb.sv  
|   └─ protocol_tb.sv  
|  
├─ testcases/  
|   ├─ alu_tests.json  
|   └─ protocol_tests.json  
|  
├─ outputs/  
|   ├─ day1_design.json  
|   ├─ day2_signals.json  
|   ├─ day3_logic.json  
|   ├─ day4_protocol.json  
|   ├─ day5_testplan.json  
|   └─ day6_results.json  
|  
├─ waves/  
|  
├─ logs/  
|   └─ issue_log.txt  
|  
└─ README.md
```

Rules & Policy (Week-1 RTL Assignment)

❖ Core Rules

- Each group works independently
- No cross-group dependency or comparison
- Focus on deep understanding, not covering everything

❖ Work Rules

- All members must understand assigned topics
- Tasks can be divided, but knowledge must be shared internally
- Maintain daily learning and experiment notes
- Produce real outputs (even if incomplete)

❖ Documentation Rules

- Maintain:
 - Learning notes
 - Experiment logs
 - Outputs
 - Issue log
- Clearly document:
 - What worked
 - What failed
 - What is not understood

❖ Issue Logging Rules (Very Important)

- Be honest about problems
- Clearly mention:
 - Errors
 - Confusions
 - Assumptions
- Do NOT hide mistakes

❖ Behavior Rules

- Do not copy without understanding
- Do not depend on other groups
- Try solving before asking
- Discuss within your group

❖ Evaluation Focus: You will be evaluated based on:

- Depth of understanding
- Clarity of explanation
- Effort and approach
- Issue identification
- Ability to think and explain

Topics Exploration and Experimental Work

❖ Day 1: ALU Design Understanding

Basic Terms (Understand First)

- Module
- Input, Output
- Opcode

Learning Topics

- What's an ALU?
- ALU operations
- Operation selection using opcode

Experimental work

- Step 1: Create git repositories
 - Connect your local git to github using ssh
 - Push your first few files there and make basic folder structure
- Step 2: Understand how a basic ALU works
- Step 3: Use systemverilog to write a basic ALU Module, edaplayground is easy to use
- Step 4: Implement two basic operations (ADD, SUB)
- Step 5: Generate Output
 - Alu.sv
- Step 6: Use python to make a simple file parser to extract json format informations

❖ Day 2: Protocol Understanding

Basic Terms (Understand First)

- Protocol
- Data Transfer
- FSM

Learning Topics

- How protocol design is different from ALU?
- Basic idea of FSM

Group A (UART)

Understand:

- TX/RX
- start/stop bits

Group B (PS/2)

Understand:

- clock + data line
- serial transfer

Group C (SPI)

Understand:

- master/slave
- clock-driven transfer

Experimental Work

- Step 1: Study protocol behavior
- Step 2: Draw basic flow
- Step 3: Define signals

❖ Day 3: Complete ALU & Signal Understanding

Basic Terms (Understand First)

- Bus
- Width
- Control Signal

Learning Topics

- 8 bit operations
- Overflow / Division issues
- Signal Roles

Experimental work

- Step 1: Write a complete ALU with all operations
- Step 2: Identify data signals vs control signals
- Step 3: Document those signal informations in json format

❖ Day 4: Verification Thinking & Design

Basic Terms (Understand First)

- Test case
- Expected output
- Edge case, corner case

Learning Topics

- What is verification?
- What needs to be tested and why?
- What are the different kinds of test(normal, boundary, invalid input)?
- How operations work internally
- Use of case and how it's suitable
- Look for edge cases that can arise (division by zero, overflow)

Experimental Work

- Step 1: Create a test plan document
- Step 2: Define Testcases , for example :
 - ADD normal
 - Overflow
 - Division by zero
- Step 3: Store test plan as json
- Step 4: Map opcode to behavior
- Step 5: Document logics and implementation clearly
- Step 6: Identify edge cases
- Step 7: Plan different coding ways to implement testplan

❖ Day 5: Protocol RTL Design & Test Planning

Basic Terms (Understand First)

- RTL

- Control path
- Datapath
- FSM design

Learning Topics

- FSM based design
- Control & Datapath interaction

Experimental Work

- Step 1: Write basic RTL for protocol
- Step 2: Define Testcases :
 - Normal
 - Edge cases
- Step 3: Store test cases in json format

❖ Day 6: Basic Testbench development

Basic Terms (Understand First)

- Testbench
- Stimulus
- Initial block

Learning Topics

- Applying inputs in sv
- Observing outputs
- What's a stimulus

Experimental Work

- Step 1: Write a simple testbench
- Step 2: Apply testcases for each feature
- Step 3: Think of ways to provide the inputs of the testcases
- Step 4: Compare expected vs actual
- Step 4: Print pass/fail result logs